

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL INTEGRADOR

CLASE 07 • NIVEL 4 - INTEGRADOR

Clase 07: Containerización Profesional con Docker y Compose

«Docker como Contenedores Estándar de Carga Marítima para Software»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)
Rol: AI Solutions Architect & Principal Software Engineer
Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 07** del **Curso 4: Taller Práctico & Proyecto Final Integrador**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.



Objetivos de la Sesión

Competencia Conceptual: Comprender imágenes, contenedores, capas, Dockerfile multi-stage, redes y volúmenes de Docker Compose.

Competencia Práctica: Empaquetar la aplicación completa (FastAPI + Streamlit + PostgreSQL) en contenedores reproducibles.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 07: Containerización Profesional con Docker y Compose

Docker elimina el famoso problema de 'en mi máquina sí funciona' empaquetando el código con todas sus dependencias.

☀ **Metáfora Central: Docker como Contenedores Estándar de Carga Marítima para Software**

Un contenedor Docker es como un contenedor de barco: no importa si va en tren o camión, su contenido viaja aislado y seguro.

Principios Teóricos y Modelo Mental

Dockerfile: La receta paso a paso para construir la imagen del contenedor.

Docker Compose: La herramienta para definir y correr aplicaciones multi-contenedor con un solo comando.

⚡ **Regla de Oro en Python**

Usa imágenes base ligeras (ej. python:3.11-slim) para reducir el tamaño y vulnerabilidades.

Arquitectura y Movimiento de Datos en Memoria

Arquitectura de contenedores aislados comunicados por red interna.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Lectura de docker-compose.yml.	Servicios definidos.
2. Evaluación	Build de imágenes para FastAPI y Streamlit.	Imágenes construidas localmente.
3. Transformación	Arranque de PostgreSQL con volumen persistente.	Base de datos en estado Healthy.
4. Retorno / Salida	Vinculación de puertos (8000, 8501, 5432) hacia el host.	Aplicación disponible en localhost.

Visualización Mental

Los contenedores son efímeros; los datos permanentes deben vivir en un 'volume'.

Código de Demostración en Python 3.10+

Configuración estándar de Dockerfile y Docker Compose:

main.py (Python 3.10+)

PEP 8 Compliant

```
DOCKERFILE_EXAMPLE = """FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]"""

print("Dockerfile de producción configurado:")
print(DOCKERFILE_EXAMPLE)
```

Análisis de Ingeniería del Código

Uso de flags --no-cache-dir y copia en capas para aprovechar la caché de Docker.

Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Contenedores con tamaños gigantescos.

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar imágenes completas basadas en Ubuntu instala compiladores innecesarios generando imágenes de más de 2 GB.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

FROM ubuntu:latest # ❌ Imagen pesada y lenta de descargar

✅ Patrón Correcto

FROM python:3.11-slim # ✅ Ligera (~150 MB) y rápida

🛡 Consejo de Resiliencia en Producción

Crea un archivo .dockerignore para evitar copiar venv, git y cachés dentro de la imagen.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 07: Containerización Profesional con Docker y Compose**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Docker como Contenedores Estándar de Carga Marítima para Software
2. Regla de Oro	Usa imágenes base ligeras (ej. python:3.11-slim) para reducir el tamaño y vulnerabilidades.
3. Gotcha Crítico	Usar imágenes completas basadas en Ubuntu instala compiladores innecesarios generando imágenes de más de 2 GB.
4. Buenas Prácticas	Crea un archivo .dockerignore para evitar copiar venv, git y cachés dentro de la imagen.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Crea un archivo `docker-compose.yml` que levante un servicio web y un servicio de Redis.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.